# Advanced Data Management

**Midterm Mock Exam**

Cycle Ingénieur · S3 · Fall 2026

**Duration:** 2 hours **Total:** 100 points
**Name:** _____

**Instructions.** Answer all three parts. *Show your reasoning* — in this exam a correct answer with no justification earns little credit, and a wrong final number with sound reasoning earns most of it. State any assumption you make. Cost is measured in **page I/Os** unless stated otherwise; a page is **8 KB**.

| Part | Topic | Points |
|---|---|---|
| 1 | Multiple choice (reason + justify) | 10 |
| 2 | Discussion | 45 |
| 3 | Implementation & hands-on (the `shopstream` database) | 45 |
| | **Total** | **100** |

## Part 1. Multiple choice — circle one and justify (10 pts)

*Each question is worth 2 points: 1 for the correct choice, 1 for a correct one- or two-line justification (a number, a formula, or a sentence). Guessing without justification scores at most 1.*

1. **External merge sort.** A file of $B = 800$ pages is sorted with $M = 21$ buffer pages. Counting the run-generation phase as one pass, how many *passes* over the data does the sort make? **[2 pts]**
   (a) 2 (b) 3 (c) 4 (d) 39
   *Justify:* _____

2. **Selectivity.** A 500,000-row table occupies 5,000 pages and has a B-tree index on `amount`. A query `WHERE amount > X` returns about **40%** of the rows. The planner will most likely choose: **[2 pts]**
   (a) an index scan, because the index is $O(\log B)$
   (b) a sequential scan, because many random heap fetches cost more than one sequential pass
   (c) an index-only scan, always
   (d) impossible to say without the exact value of X
   *Justify:* _____

3. **Buffer replacement.** A buffer pool has **3** frames and uses **LRU**, starting empty. Pages are requested in this order:
   $$A,\ B,\ C,\ A,\ B,\ D,\ A,\ B$$
   How many of the 8 requests are *hits*? **[2 pts]**
   (a) 2 (b) 3 (c) 4 (d) 5
   *Justify (show the eviction):* _____

4. **B$^+$ tree lookup.** A B$^+$ tree indexes 2,000,000 rows with an average fanout of 200. About how many *page reads* does a single point lookup by key take (including fetching the row from the heap)? **[2 pts]**
   (a) 3 (b) 4 (c) 5 (d) 21
   *Justify:* _____

5. **Join cost.** A block nested-loop join has outer relation $B_R = 300$ pages, inner $B_S = 3{,}000$ pages, and $M = 32$ buffer pages. Using $B_R + \lceil B_R/(M-2) \rceil \cdot B_S$, the page-read cost is: **[2 pts]**
   (a) 9,300 (b) 30,300 (c) 900,300 (d) 3,300
   *Justify:* _____

## Part 2. Discussion (45 pts)

*Answer in clear prose. Depth and correctness matter more than length; a focused half-page beats two vague pages.*

1. **Why the DBMS runs its own buffer pool.** The operating system already caches file pages in memory. Give **three** concrete reasons a database manages its *own* buffer pool on top of that, and for each say what would break or slow down if it relied on the OS alone. Connect at least one reason to durability. **[7 pts]**

2. **Why we count pages, not instructions.** Explain why "number of pages read" is the cost metric throughout Chapters 2–5, rather than CPU instructions. Then give one concrete situation where this approximation *misleads* you — where CPU cost actually dominates — and say how you would notice it in an `EXPLAIN ANALYZE`. **[7 pts]**

3. **"Index everything."** A colleague adds a B-tree index on *every* column of a large, write-heavy table "to be safe." Explain the full consequences — for reads, for writes, for disk space, and for the optimizer. Then explain, using the idea of *selectivity*, why some of those indexes will never even be used by the planner. Finally, state how you would decide which indexes to keep. **[8 pts]**

4. **Three joins, kept on purpose.** Compare nested-loop, sort-merge, and hash join. For *each*, give one situation where it is clearly the best choice and one where it is a disaster. Then answer: why does a database keep all three instead of always using the hash join? **[8 pts]**

5. **The hardest part of optimization.** Defend the claim that *cardinality estimation* is the hardest and most important job of the optimizer. Explain precisely how a wrong row-count at a leaf of the plan propagates upward and damages the final plan. Give **two independent** reasons estimates go wrong, and the fix for each. **[8 pts]**

6. **Sequential flooding and the steal policy.** (a) Explain *sequential flooding*: what it is, why plain LRU is vulnerable to it, and how a real system defends against it. (b) The buffer manager uses a **steal** + **no-force** policy. Define both terms, and explain why this choice — made purely for performance — forces the database to keep a log. **[7 pts]**

## Part 3. Implementation & hands-on (45 pts)

---

### The database: `shopstream`

You inherit a production PostgreSQL database for a shopping/telemetry platform. It is *not* the tidy `adm_lab` from the labs — it is large, skewed, and under-maintained.

**Schema (simplified):**

- `users(user_id PK, email, country, city, segment, signup_ts)` — **2,000,000** rows, ≈ **30,000** pages. *city determines country* (every Casablanca user is in MA).
- `products(product_id PK, title, price, category)` — 50,000 rows.

- `events(event_id PK, user_id, event_type, ts, product_id, payload jsonb, device)` — **50,000,000** rows, ≈ **1,000,000** pages (≈ 8 GB), ≈ 50 rows/page.

**Nasty properties:**

- `event_type` is **heavily skewed**: `'view'` ≈ 90%, `'cart'` ≈ 9.5%, `'purchase'` ≈ 0.5%.
- `payload` is large JSON, stored out-of-line (**TOAST**).
- `country` and `city` are strongly **correlated**.
- The **only** indexes are the primary keys. No index on `user_id`, `ts`, `event_type`, `email`, `country`, or `city`.
- `ANALYZE` last ran when `events` had **5M** rows (it now has 50M) — the statistics are **stale**.
- Config: `shared_buffers = 4 GB` (≈ **500,000** pages); `work_mem = 64 MB` (≈ **8,000** pages). Sequential throughput ≈ 500 MB/s.
- Today is **2026-10-12**.

---

1. **Diagnose a bad plan.** This query takes **13 minutes**:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT u.country, count(*)
FROM users u JOIN events e ON e.user_id = u.user_id
WHERE u.city = 'Casablanca' AND e.event_type = 'purchase'
GROUP BY u.country;
```

```
HashAggregate  (actual time=781204 ms)
  ->  Nested Loop  (cost=.. rows=50)  (actual rows=1240000)
        ->  Seq Scan on users u  (rows=40)  (actual rows=200000)
              Filter: (city = 'Casablanca')
        ->  Seq Scan on events e  (per outer row)
              Filter: (user_id = u.user_id AND event_type = 'purchase')
```

   (a) Identify the single node whose estimate is most wrong, and give the number. (b) State the **two** root causes. (c) Give the exact commands you would run to fix the plan, and say which join the optimizer should choose instead. **[6 pts]**

2. **Storage & columnar.** (a) Confirm that a full sequential scan of `events` reads ≈ 1,000,000 pages, and compute how long that cold scan takes at 500 MB/s. (b) You only need the `ts` column (8 bytes) for an analysis. Write the `COPY` that exports it to Parquet and the DuckDB query that counts rows per day, and explain why this is far cheaper than the Postgres scan. (c) Write the two commands (using `pg_relation_size` and `pageinspect`) you would run to *verify* the rows-per-page figure empirically. **[6 pts]**

3. **Index design.** For each query pattern, give the index you would create (write the `CREATE INDEX`), name its type, and justify it in one line. Then identify the **one** pattern for which *no* index will help, and prove it with a selectivity number. **[8 pts]**

   (a) `WHERE email = $1` (unique lookup)

   (b) `WHERE ts BETWEEN $1 AND $2` (date range)

   (c) `WHERE event_type = 'purchase'`

   (d) `WHERE event_type = 'view'`

   (e) `WHERE lower(email) = $1`

   (f) `JOIN ... ON e.user_id = u.user_id`

4. **External sort & join cost.** A query must sort all of `events` (1,000,000 pages) by `ts`, with `work_mem` giving $M \approx 8{,}000$ buffer pages. (a) Compute the number of initial sorted runs, the number of passes, and the total I/O. (b) A different query joins `events` to `users` on `user_id` with no useful index. Which join algorithm should the optimizer pick, and why? Estimate its I/O, and state explicitly whether the build side fits in `work_mem`. **[8 pts]**

5. **Estimation failures.** (a) For `WHERE country='MA' AND city='Casablanca'`, with $\text{sel}(country='MA') = 0.2$ and $\text{sel}(city='Casablanca') = 0.1$, compute the planner's independence estimate and the *true* count, explain the gap, and give the command that fixes it. (b) Contrast `event_type='view'` with `event_type='purchase'`: give the selectivity and resulting row count for each, and say which one an index would serve and which one the planner will scan — with the reason. **[8 pts]**

6. **Buffer behaviour.** With `shared_buffers` $\approx$ 500,000 pages and the `events` scan touching 1,000,000 pages (cold): (a) what hit ratio do you expect for one full scan, and why? (b) Under plain LRU, what happens to the pages other queries were using, and what does PostgreSQL actually do to prevent it? (c) Write the `pg_buffercache` query that shows which tables currently occupy the pool. **[5 pts]**

7. **Write the SQL (lab style).** (a) Write one query: the number of `purchase` events per country in the **last 30 days**. (b) Write a single `generate_series` statement that inserts 1,000,000 synthetic `events` whose `event_type` follows the skew above ($\approx$90% `view`, $\approx$9.5% `cart`, $\approx$0.5% `purchase`). **[4 pts]**

— End of exam. Good luck! —